



Home



My Network



Jobs



Messaging



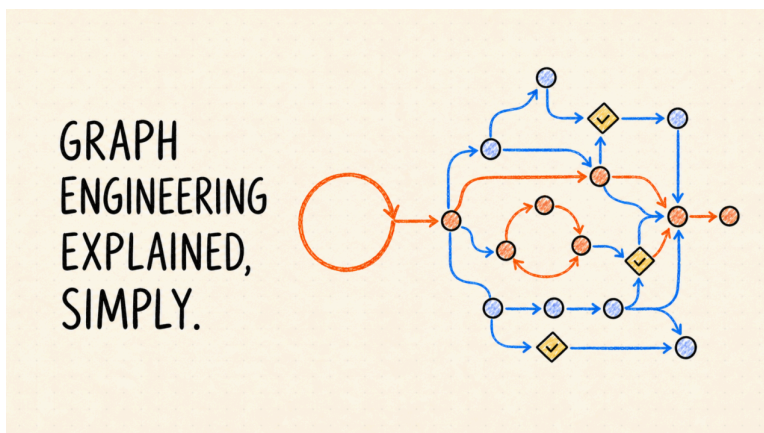
Notifications



Me



For Bus



مهندسی گراف چیه و چرا سیستم‌های چندعاملی به آن نیاز دارند؟



Mohamad Reza Moghadasi ✓
Senior DevOps & SRE Consultant



September 15, 2026

تا همین چند ماه پیش هر وقت پست یا مقاله‌ای درباره «ایجنت‌های خودمختار» (Autonomous Agents) می‌دیدم تصور می‌کردم، چند تا ایجنت را داخل یک سرور رها می‌کنیم، صورت مسئله را به آن‌ها می‌گیریم و خودشان مثل یک تیم کامل استارت‌آپی از پس همه‌چیز برمی‌آیند.

اما هر کسی که یک بار سعی کرده باشه چنین سیستمی را برای یک کار واقعی پروداکشن (مثلاً بررسی خودکار پول‌ریکوئست‌ها، امور مالی یا تحلیل داده‌های پیچیده) بالا بیاورد، خیلی زود با این حقیقت مواجه شده: حلقه‌های بی‌پایان، توهم‌های پرهزینه، گم شدن کانتکست و خروجی‌هایی کاملاً غیرقابل پیش‌بینی.

اینجاست که مفهومی به اسم «مهندسی گراف» (Graph Engineering) وسط می‌آید. مفهومی که شاید در نگاه اول اسمش شبیه مباحث سنگین پایگاه داده به نظر برسه، اما در عمل، پلی است میان آشفتگی هوش مصنوعی و نظم مهندسی نرم‌افزار.

در این یادداشت می‌خواهم خیلی ساده و با مثال‌های واقعی بررسی کنیم که مهندسی گراف اصلاً چیه، چرا با «نالچ گراف» فرق داره و چطور ترکیب آن با ابزارهایی مثل Google ADK و دیپ‌لرنیگ می‌تواند چالش‌های توسعه سیستم‌های Multi-Agent را به یک معماری تمیز تبدیل کنه.

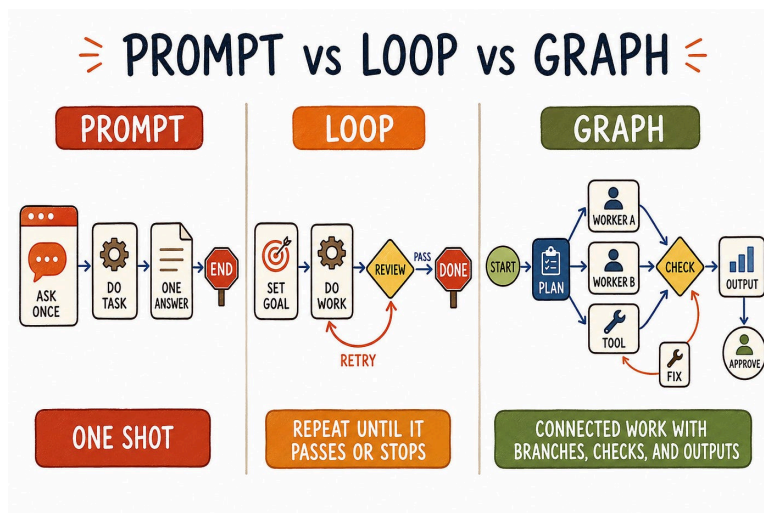
بخش اول: یک‌بار برای همیشه؛ این اصطلاحات گیج‌کننده یعنی چه؟

وقتی وارد دنیای ایجنت‌ها می‌شین، چند واژه مدام جلوی چشمتون می‌آید: Harness، Graph و Loop. بیایید این سه تا را خیلی شفاف از هم تفکیک کنیم:

بستر یا مهار (Harness): تصورتان از Harness باید تمام ابزارها و لایه‌های حفاظتی دور مدل باشه؛ یعنی کدهای فراخوانی ابزار (Tools)، سیستم حافظه (Memory) و گاردریل‌هایی که جلوی خطاهای مدل را می‌گیرند. مدل بدون Harness یک متن‌ساز ساده است؛ با Harness تبدیل به ایجنت می‌شود.

حلقه (Loop): چرخه‌ای که ایجنت در آن فکر می‌کنه: «ورودی را خواندم ➡ چه ابزاری صدا بزنم؟ ➡ خروجی ابزار چی شد؟ ➡ آیا کار تمام است یا دوباره باید فکر کنم؟» این حلقه آن قدر می‌چرخه تا مدل به هدف برسه.

گراف (Graph): حالا اگر چند تا از این ایجنت‌ها یا توابع برنامه‌نویسی را کنار هم بگذاریم، رابطه بین آن‌ها چطور تعریف می‌شه؟ این دقیقاً گراف است؛ درست شبیه به یک «چارت سازمانی» شفاف در یک شرکت.



یک اشتباه رایج: مهندسی گراف با نالچ گراف فرق دارد!

خیلی‌ها وقتی کلمه گراف را کنار هوش مصنوعی می‌شنوند، ذهنشون سریع میره سراغ Knowledge Graph یا گراف دانش؛ یعنی موجودیت‌ها، رابطه‌ها و پایگاه‌های داده‌ای مثل Neo4j.

اما نکته اینجاست:

گراف دانش (Knowledge Graph) روی «داده‌ها» تمرکز دارد؛ در حالی که مهندسی گراف (Graph Engineering) روی «رفتار و جریان کنترل» (Behavior & Control Flow) تمرکز می‌کند.

یعنی در مهندسی گراف، سوال ما این است: داده با چه ترتیبی وارد شود؟ اگر فلان تابع شکست خورد چه اتفاقی بیفتد؟ خروجی مرحله الف چطور به مرحله ب تزریق شود؟

چرا به جای Agent Swarm به گراف نیاز داریم؟

الگوی دیگری به اسم Agent Swarm وجود داره که در آن ایجنت‌ها مثل گروهی از زنبورها هستند؛ هر کدام شخصیت و وظیفه‌ای دارند و با هم آزادانه گفتگو می‌کنند تا مسئله حل شود.

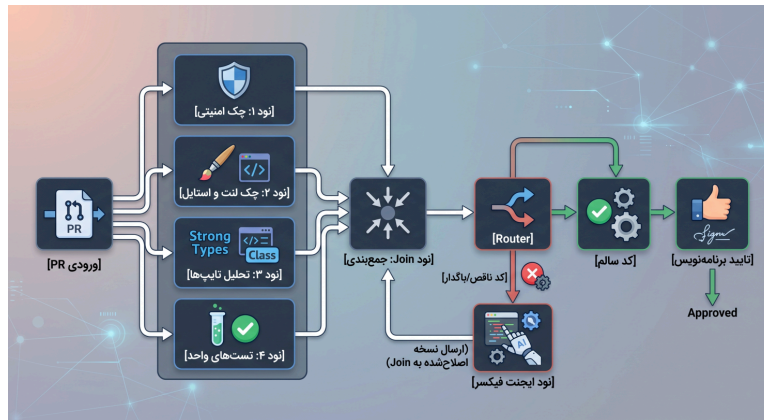
این الگو برای کارهای خلاقانه، طوفان فکری یا مسائلی با ابهام بالا عالی است. اما بیا این صادق باشیم: آیا حاضر هستین فرآیند حساس دیپلوی نرم‌افزار، حسابداری شرکت یا بررسی PR تیمتان را به یک گفتگوی آزاد بین چند ایجنت بسپارید؟ قطعاً نه!

در سناریوهایی مثل بررسی یک پول‌ریکویست (PR Review):

فرض کنیم یک ایجنت واحد داریم. این ایجنت در یک حلقه `while` قرار می‌گیرد. به او کل کد `PR` را می‌دهیم و می‌گوییم: «بررسی‌اش کن، اگر ایراد داشت درستش کن، اگر خوب بود تایید کن.» ایجنت در هر دور حلقه باید هم امنیت را چک کند، هم فرمت را ببیند، هم منطق بیزینس را ارزیابی کند و هم تصمیم بگیرد. **نتیجه چیست؟** کانتکست ایجنت بیش از حد شلوغ می‌شه، ممکن است در دور دوم بررسی امنیتی یادش بره چه توصیه‌ای کرده بود، مدام بین نقش‌های مختلف سردرگم بشه، و اگر در اصلاح کد به مشکلی بخوره، وارد یک چرخه تکراری بی‌پایان (`Loop`) می‌شه که هزینه توکن‌های شما را هدر می‌دهد و کار متوقف می‌شه.

اما در «مهندسی گراف» (Graph Engineering) ماجرا چطور است؟

در مهندسی گراف، شما کار را به یک توپولوژی از گره‌ها و یال‌ها تبدیل می‌کنید:



سه دلیلی که این را تبدیل به یک گراف واقعی می‌کند:

1. **انشعاب و همزمانی (Fan-out & Join):** یک حلقه ساده توانایی اجرای موازی ندارد؛ مرحله به مرحله پیش می‌رود. اما در گراف، سیستم به ۴ گره مستقل شاخه می‌زند. این ۴ ایجنت همزمان و با حداکثر سرعت که در تحلیل می‌کنند. Join مثل یک سد هماهنگی (Barrier) منتظر می‌ماند تا همه تمام شوند و داده‌ها را یکجا تلفیق کند.
2. **ایزوله بودن کانتکست هر گره:** ایجنتی که نود ۱ (امنیت) را چک می‌کند، نیازی ندارد درگیر استایل و تورفتگی کد باشد؛ کانتکست او تمیز و متمرکز است.
3. **مسیرهای هدایت‌شده (Explicit Routing):** اگر کد ایراد داشت، به ایجنت فیکس می‌رود، ولی فیکس سرخود هر کاری نمی‌کند؛ خروجی فیکس روی یک پال مشخص مجدداً به نقطه مشخصی از گراف برمی‌گردد، نه اینکه در حلقه برای خودش تصمیم جدید بگیرد.

سه الگوی اصلی برای طراحی گردش کار ایجنت‌ها

وقتی با فریم‌ورک‌هایی مثل Google Agent Development Kit (یا همان ADK) کار می‌کنید، فلوی کاری شما معمولاً ترکیبی از ۳ الگوی زیر است:

1. **عامل‌های ترتیبی (Sequential Agents):** مثل خط مونتاژ کارخانه. یک ایجنت داده را دریافت/پول می‌کند، به دومی می‌دهد تا پاکسازی شود، سومی تحلیل را انجام می‌دهد و چهارمی خلاصه را می‌نویسد. ترتیب ثابت و غیرقابل تغییر است.
2. **عامل‌های موازی (Parallel Agents):** مثل مدیری که یک پروژه را صبح به سه‌تا دولوپر می‌دهد تا هر کدام بخشی از API را بزنند. کارها همزمان انجام می‌شود و زمان اجرای کل تسک به اندازه کمترین تسک کاهش پیدا می‌کند.
3. **عامل‌های حلقه‌ای (Loop Agents):** شبیه برنامه‌نویسی که یک باگ را آن‌قدر تست و ریفت‌کور می‌کند تا بالاخره ارور برطرف شود. تسک مدام تکرار می‌شود تا شرط خروج (مثلاً پاس شدن تست یا رسیدن به سقف ۵ تلاش) برقرار گردد.

ایجنت‌ها چطور با هم حرف می‌زنند؟

وقتی چند ایجنت در سیستم داریم، مکانیزم تبادل داده بین آن‌ها فوق‌العاده حیاتی است. معمولاً سه سبک اصلی داریم:

وضعیت اشتراکی (Shared Session State): تصور کنید روی دیوار اتاق، یک تخته وایت‌برد بزرگ نصب است. ایجنت اول خروجی‌اش را روی تخته می‌نویسد و می‌رود. ایجنت دوم وارد می‌شود، تخته را می‌خواند، تغییراتش را اضافه می‌کند. هیچ مکالمه مستقیمی رخ نداده، اما داده به اشتراک گذاشته شده است.

تفویض اختیار هوشمند (LLM-driven Delegation): یک ایجنت در نقش مدیرعامل (CEO) می‌نشیند. درخواست کاربر را می‌شنود و تشخیص می‌دهد: «این موضوع حقوقی است، پس کار را محول می‌کنم به ایجنت امور قرارداده‌ها».

ایجنت به عنوان ابزار (Agent as a Tool): تفاوت این الگو با حالت قبلی مثل تفاوت کارمند رسمی با مشاور بیرونی است. ایجنت والد هر وقت لازم داشته باشد، یک ایجنت دیگر را مثل یک تابع صدا می‌زند. مثلاً ایجنت تحلیلگر متن، هر جا به یک معادله ریاضی برسد، ایجنت ماشین‌حساب را موقتاً فراخوانی می‌کند و نتیجه را می‌گیرد.

از تئوری به عمل: وقتی GraphRAG وارد بازی می‌شود

بیا یه یک پله عمیق‌تر برویم. در سیستم‌های سنتی بازیابی اطلاعات (RAG)، متن به بردار (Vector) تبدیل می‌شود و بر اساس شباهت معنایی جستجو می‌کنیم. اما فرض کنید روی یک سناریوی اضطراری پیش آمده: چندین نفر توی منطقه‌ای گرفتار شده‌اند و شما می‌خواهید بدانید: «چه کسی می‌تواند سوختگی دست یکی از اعضا را درمان کند؟»

در **RAG سنتی**: شما واژه «درمان» یا «سوختگی» را جستجو می‌کنید و سیستم نهایتاً متنی پیدا می‌کند که توییش نوشته «آموزش‌های پزشکی». اما نمی‌داند این مهارت دقیقاً متعلق به کدام شخص است، آن شخص الان کجاست و با فرد مصدوم چه فاصله‌ای دارد!

در **GraphRAG**: سیستم ابتدا با امیدینگ، مفهوم مهارت پزشکی را پیدا می‌کند، اما بعد از آن روی یال‌های گراف حرکت (Traverse) می‌کند؛ می‌بیند این مهارت به دکتری به نام علی وصل است و علی الان آتا خیابون بالاتر هست و می‌تواند به مصدومی که در همان حوالی است کمک کند.

این یعنی تلفیق قدرت ادراک مدل‌های زبانی با منطق رابطه‌ای گراف‌ها.

چالش پیاده‌سازی و یک ترفند مهم معماری

معمولاً برای پیاده‌سازی GraphRAG تیم‌ها دو تا دیتابیس می‌آورند: یکی دیتابیس وکتوری/رابطه‌ای (مثل Postgres) و یکی دیتابیس گرافی (مثل Neo4j). نتیجه؟ دردهای بی‌پایان همگام‌سازی داده، تأخیر شبکه و شکستن ترنزشن‌ها.

رویکرد مدرن‌تری که این روزها در اکوسیستم‌های ابری (مثلاً با Google Cloud Spanner Graph) پیاده می‌شود، استفاده از **دیتابیس یکپارچه** است. یک نکته بسیار جالب در این معماری، بردن پردازش هوش مصنوعی به داخل دیتابیس است (با قابلیت‌هایی مثل ML.PREDICT).

فرض کنید در دیتابیس خود متن رزومه ۱۰۰ نفر را دارید و می‌خواهید آن‌ها را به بردار (امیدینگ) تبدیل کنید تا بتوانید جستجوی معنایی انجام دهید. در حالت عادی، سیستم شما این مسیر پرپیچ‌وخم را طی می‌کند:

1. یک‌اند به دیتابیس وصل می‌شود و متن ۱۰۰ رزومه را از شبکه رد می‌کنه و میاره داخل رم (RAM) خودش.
2. یک‌اند تک‌تک متن‌ها را در قالب ۱۰۰ درخواست اینترنتی (HTTP) برای API هوش مصنوعی (مثلاً مدل‌های گوگل یا OpenAI) می‌فرستد و منتظر می‌ماند.
3. پاسخ‌ها برمی‌گردند به یک‌اند (حجم عظیمی از آرایه‌های عددی اعشاری).
4. یک‌اند دوباره یک کوئری به دیتابیس می‌زند و تمام این بردارها را به دیتابیس برمی‌گرداند تا ذخیره شوند!

مشکل این سناریو چیست؟ حجم عظیمی از داده مدام بین دیتابیس، سرور شما و API بیرونی در حال رفت‌و برگشت است. اگر تعداد داده‌ها زیاد شود، سیستم کند می‌شود، هزینه ترافیک بالا می‌رود و بار پردازشی سرور یک‌اند بیهوده اشغال می‌شود.

روش مدرن در پایگاه داده‌هایی مثل Spanner (یا ML.PREDICT):

مثل این است که به جای بردن اسناد به اداره دیگر، کارشناس آن اداره را بیاورید داخل اتاق بایگانی خودتان بنشانید!

شما یک کوئری SQL ساده می‌نویسید:

```
SELECT skill_text, embeddings
FROM ML.PREDICT(MODEL TextEmbeddingModel, TABLE Skills)
```

موتور خود دیتابیس، مستقیماً با مدل هوش مصنوعی در ارتباط است. یعنی:

- داده‌ها اصلاً دیتابیس را ترک نمی‌کنند و وارد سرور یک‌اند شما نمی‌شوند.
- تبدیل متن به بردار، ذخیره‌سازی، و حتی محاسبه فاصله بردارها (شباهت معنایی) همگی در همان لایه دیتابیس و در چند میلی‌ثانیه انجام می‌شوند.
- نتیجه؟ مصرف حافظه سرور شما نزدیک به صفر می‌شود و تأخیر شبکه (Latency) بین لایه‌ها به کلی از بین می‌رود.

جمع‌بندی: چه زمانی باید سراغ مهندسی گراف برویم؟

اگر پروژه شما قرار است در حد یک دستیار چت ساده، خلاصه‌ساز مقالات یا یک بات ایده‌پردازی باشد، خودتان را با گراف‌ها درگیر نکنید، همان پرامپت‌های استاندارد و چرخه‌های ساده (Loop) کارتان را راه می‌اندازند.

اما اگر در حال ساخت سیستمی هستید که:

1. چند گام مشخص و وابسته به هم دارد،
2. خطای تصادفی در آن مساوی با شکست کل فرآیند است،
3. و نیاز به رهگیری دقیق (Traceability) دارید تا بفهمید چرا و در کدام گره تصمیمی اشتباه گرفته شده،

مهندسی گراف انتخاب نهایی شماست. وقت آن رسیده که از توهم «مدل‌ها خودشان همه‌چیز را مدیریت می‌کنند» بیرون بیاوریم و کنترل جریان برنامه‌هایمان را مثل یک مهندس نرم‌افزار واقعی دوباره به دست بگیریم.

منابع مفید:

<https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications><https://www.anthropic.com/engineering/building-effective-agents><https://docs.langchain.com/oss/python/langgraph/workflows-agents><https://arxiv.org/abs/2404.16130>

اگر شما هم در توسعه پروژه‌های خود با چالش‌های سیستم‌های چندعاملی یا ترکیب پایگاه‌های داده گرافی با LLM‌ها دست‌وپنجه نرم کرده‌اید، خوشحال می‌شم تجربیاتتون را بشونم.

Comments

11



Like



Comment



Share

Add a comment...



No comments, yet.

Be the first to comment.

[Start the conversation](#)

Mohamad Reza Moghadasi

Senior DevOps & SRE Consultant

✓ Following

About

Professional Community Policies

Privacy & Terms

Sales Solutions

Safety Center

Accessibility

Careers

Ad Choices

Mobile

Talent Solutions

Marketing Solutions

Advertising

Small Business



Questions?

Visit our Help Center.



Manage your account and privacy

Go to your Settings.



Recommendation transparency

Learn more about Recommended Content.

Select Language

English (English)

